

RELAZIONE DI PROGETTO

HackHub

Piattaforma per la gestione di hackathon

Descrizione generale del progetto

Autori

Maurizio Crescenzi · Luca Gasparretti

UNICAM — Anno Accademico 2025/2026

Indice

Introduzione	3
Ruoli e livelli di accesso	3
Ciclo di vita di un hackathon	4
Architettura	5
Stack tecnologico	5
Sicurezza	6

Introduzione

HackHub è un'applicazione web che copre l'intero ciclo di vita di un hackathon, dalla creazione dell'evento fino alla proclamazione del vincitore. La piattaforma mette a disposizione un ambiente centralizzato in cui istituire e amministrare le competizioni; gli utenti, dal canto loro, possono costituire nuovi team, entrare a far parte di gruppi già esistenti e caricare i propri elaborati, con la facoltà di aggiornarli fino alla scadenza prevista.

Il sistema comprende inoltre un meccanismo di valutazione affidato ai giudici, articolato in punteggi e feedback, e attribuisce agli organizzatori il compito di designare il team vincitore. Completano il quadro le funzioni di supporto: l'affiancamento da parte dei mentori, la pianificazione di call e la segnalazione di eventuali violazioni del regolamento.

Ruoli e livelli di accesso

La piattaforma prevede diverse tipologie di utenza, ognuna con responsabilità e permessi propri. I ruoli si articolano in due famiglie: quella dei ruoli statici, attribuiti in fase di registrazione, e quella dei ruoli dinamici, che descrivono il modo in cui un ruolo statico interagisce con il sistema.

Ruolo statico	Ruoli dinamici assumibili
Staff	Organizzatore, Giudice, Mentore
Utente	Team leader, Membro del team

Visitatore

Il visitatore è chi accede al sito senza autenticarsi: può consultare liberamente le informazioni pubbliche relative agli hackathon.

Utente registrato

Una volta autenticato, l'utente registrato può dare vita a un nuovo gruppo invitando altri iscritti alla piattaforma oppure accettare un invito ed entrare a far parte di un team già esistente.

Team leader

Chi crea un team ne diventa il team leader e ne amministra le attività principali: l'iscrizione a un hackathon e l'eventuale ritiro, la gestione degli inviti rivolti a nuovi membri e la modifica delle informazioni relative al gruppo.

Membro del team

Il membro del team, per le competizioni a cui prende parte, provvede all'invio della sottomissione di progetto e può aggiornarla con eventuali modifiche, purché entro la scadenza stabilita dall'evento.

Staff

Il membro dello staff è il personale assegnabile a uno specifico hackathon, nell'ambito del quale può ricoprire i ruoli di organizzatore, giudice e/o mentore.

Mentore

Il mentore affianca i team durante lo svolgimento dell'evento: attraverso la piattaforma ne monitora l'andamento delle attività e, qualora riscontri violazioni del regolamento da parte di un team, le segnala all'organizzatore, al quale spettano le decisioni opportune.

Giudice

Il giudice ha visibilità su tutte le sottomissioni dei team relative all'evento a cui è stato assegnato e per ciascuna rilascia una valutazione composta da un punteggio numerico compreso tra 0 e 10 e da un breve giudizio scritto che ne motiva l'esito.

Organizzatore

L'organizzatore crea nuovi hackathon definendone tutte le informazioni essenziali e può aggiungere ulteriori mentori anche dopo la creazione dell'evento. Quando il giudice ha valutato tutte le sottomissioni, proclama l'unico team vincitore della competizione.

Ciclo di vita di un hackathon

Ogni hackathon attraversa quattro stati distinti, ciascuno dei quali determina le funzionalità disponibili per i diversi attori:

Stato	Descrizione	Azioni
REGISTRATION	Iscrizioni aperte	I team si iscrivono all'evento
ONGOING	Competizione in corso	I team lavorano e caricano le sottomissioni
EVALUATION	Valutazione in corso	Il giudice valuta le sottomissioni dei team
COMPLETED	Evento concluso	L'organizzatore proclama il vincitore

Il passaggio da uno stato al successivo avviene in automatico, sulla base delle date di inizio e di fine assegnate all'hackathon. Fa eccezione la sola transizione da EVALUATION a COMPLETED, che si compie nel momento in cui l'organizzatore proclama il vincitore.

Architettura

Il sistema adotta un'architettura a tre livelli nettamente separati, a garanzia di modularità, manutenibilità e scalabilità:

Presentation layer — Frontend

Il livello di presentazione dialoga con il backend esclusivamente attraverso chiamate HTTP alle API REST esposte dal server.

Business logic layer — Backend

Il livello di logica elabora le richieste provenienti dal frontend, valida i dati, applica le regole di business e presidia la sicurezza.

Data layer — Persistenza

Il livello di persistenza è affidato a Spring Data JPA con database H2 in memoria. La modalità in-memory semplifica la predisposizione dell'ambiente di sviluppo e di test, eliminando la necessità di installare e configurare un database esterno.

Stack tecnologico

Backend — Java e Spring Boot

Il backend è realizzato in Java 21 con il framework Spring Boot, del quale il progetto impiega i seguenti moduli:

- Spring Web: gestione delle API REST tramite controller annotati con `@RestController`; ogni endpoint è associato a un percorso HTTP e a uno dei metodi standard (GET, POST, PUT, DELETE), nel rispetto dei principi architetturali REST.
- Spring Security: gestione dell'autenticazione e dell'autorizzazione.
- Spring Data JPA: astrazione del livello di persistenza tramite repository tipizzati; consente di interagire con il database per mezzo di un ORM, con Hibernate come implementazione JPA sottostante.
- H2 Database: database relazionale in memoria.

La build e la gestione delle dipendenze del backend sono affidate ad Apache Maven, che provvede alla risoluzione delle librerie, alla compilazione del codice sorgente e al packaging dell'applicazione.

Frontend — Angular

Il frontend è una Single Page Application realizzata con Angular 21. L'architettura di Angular si fonda su componenti riutilizzabili, ognuno dei quali incapsula template HTML, logica TypeScript e stili SCSS responsive.

Sicurezza

La sicurezza dell'applicazione è presidiata tanto sul frontend quanto sul backend.

Autenticazione JWT

L'accesso alle funzionalità protette della piattaforma è regolato da un sistema di autenticazione basato su JWT (JSON Web Token). Le richieste verso funzionalità protette prive di autenticazione ricevono una risposta con codice HTTP 401 (Unauthorized).

Autorizzazione basata sui ruoli

Il backend implementa un controllo degli accessi basato sui ruoli (RBAC, Role-Based Access Control). Il ruolo è custodito nei claims del token JWT, insieme all'email in qualità di subject: in questo modo ogni endpoint dell'API REST può essere protetto da specifiche annotazioni di Spring Security che ne regolano l'autorizzazione. Gli accessi non autorizzati ricevono una risposta con codice HTTP 403 (Forbidden).

CORS

Per evitare che il backend sia raggiungibile da origini arbitrarie, la configurazione CORS ammette esclusivamente le richieste provenienti dalla SPA Angular (<http://localhost:4200>).

Protezione delle password

Le password non vengono conservate in chiaro: sono sottoposte a hashing con bcrypt, messo a disposizione da Spring Security, combinando una secret (pepper) definita in fase di configurazione con il salt gestito in automatico dalla libreria stessa.

ORM e prevenzione delle SQL injection

L'adozione dell'ORM Spring Data JPA (Hibernate) consente di operare sui dati tramite oggetti anziché query scritte a mano. Ne deriva l'uso intrinseco di query parametrizzate, che escludono la concatenazione diretta degli input utente nel codice SQL e neutralizzano alla radice il rischio di attacchi SQL injection.

Guard

Sul frontend, apposite route guard di Angular impediscono agli utenti non autenticati, o privi del ruolo richiesto, di raggiungere le pagine riservate.